Arbiter: Release-Time Enforcement of Query Prices in Data Markets

Extended Version: Full Proofs, Additional Results, and the Extension Campaign

Zaheer Ahmad Gondal, Xiaofeng Ding, Attiq Ur Rehman, and Jamil Ahmad Memon
Huazhong University of Science and Technology, Wuhan, China

This document accompanies the PVLDB submission of the same title and is part of its artifact repository. Lemma, theorem, proposition, section, equation, and figure numbers refer to the paper, and the tables of this document carry their own numbers. Every number below comes from `scripts/facts.json` (audited, closure, hardening, and revision campaigns) or `scripts/facts_step2.json` (extension campaign), which record the evidence file that produced it, and every table is generated from the evidence files by `scripts/make_floats.py` or `scripts/make_step2.py`.

1 Notation

A snapshot at epoch e is a sparse system $\mathbf{A}_e\mathbf{p} \le \mathbf{b}_e$ with rows $\mathbf{a}_j^\top \mathbf{p} \le b_j$ for $j = 1, \dots, m$ over n asset prices. The encoded feasible set is $\mathcal{C}_e = \{\mathbf{p} : \mathbf{A}_e\mathbf{p} \le \mathbf{b}_e\}$, and the maximum violation is $\rho_e(\mathbf{p}) = \max(0, \max_j\{\mathbf{a}_j^\top \mathbf{p} - b_j\})$ (Equation 2). The residual of row j is $\delta_j(\mathbf{p}) = \mathbf{a}_j^\top \mathbf{p} - b_j$. Active-row repair (Section 5.2) starts each sweep by computing every residual, orders the rows with $\delta_j > \varepsilon$ by descending residual, and visits at most B of them. A visited row that is still violated receives the update of Equation 5,

$$\mathbf{p} \leftarrow \mathbf{p} - \eta \, \frac{\delta_j(\mathbf{p})}{\|\mathbf{a}_j\|_2^2} \, \mathbf{a}_j, \qquad 0 < \eta \le 1.$$

Repair stops when no row is violated, when a sweep changes nothing, or after K sweeps. We write $\mathrm{dist}(\mathbf{p}, \mathcal{C}) = \min_{\mathbf{c}\in\mathcal{C}} \|\mathbf{p} - \mathbf{c}\|_2$, $\alpha = \max_j \|\mathbf{a}_j\|_2$, $\omega = \min_j \|\mathbf{a}_j\|_2$, and $x^+ = \max(x, 0)$.

2 Full Proofs

2.1 Rule soundness

A bag is a multiset of tuples. Selection σ_ϕ keeps each tuple that satisfies ϕ with its multiplicity, projection π_X maps each tuple to its restriction to the columns X and keeps multiplicities, and $\uplus$ adds multiplicities. A tuple whose predicate column is NULL satisfies no interval or category predicate under SQL's three-valued logic, so it belongs to none of the selections below. A grouped aggregate with grouping column g and predicate ϕ returns, for each value v of g that occurs in $\sigma_\phi(T)$, one row $(v, \mathrm{agg}(G_v))$, where the group G_v is the bag of tuples of $\sigma_\phi(T)$ whose value of g is v. SQL places all tuples whose value of g is NULL in one group, so the groups partition $\sigma_\phi(T)$. COUNT$(*)$ of a group is its size, and SUM(m) adds the group's non-NULL values of m and is NULL when there are none.

Lemma 4.1 (rule soundness). *On every instance of T, the answers of the parts of an emitted decomposition (t, l, r) determine* ans(t)*: by bag union for row sets, and by addition for* COUNT *and* SUM*, group by group for grouped aggregates, where a part whose* SUM *is NULL because no row contributes a value, or a group missing from one part, counts as zero, and the target's value is NULL or missing exactly when both parts lack a value. For an emitted derivation (d, s),* ans(s) *determines* ans(d) *through the operations that R2 to R6 state.*

Proof. Rule R1. The three queries share a surface, so they read the same table T, return the same output list X, or the same grouping column, aggregate, and measure, and filter on the same column c. The predicate of q_t is the disjoint union of the predicates of q_l and q_r: either one split point s with $a < s < b$ divides the interval $[a, b)$ of q_t into $[a, s)$ and $[s, b)$, or the category set of q_t is the disjoint union of the category sets of q_l and q_r. A tuple u with a non-NULL value of c therefore satisfies ϕ_t if and only if it satisfies exactly one of ϕ_l and ϕ_r, and a tuple with a NULL value satisfies none of the three. Hence $\sigma_{\phi_t}(T) = \sigma_{\phi_l}(T) \uplus \sigma_{\phi_r}(T)$, with every multiplicity of T counted in exactly one bag on the right. For row sets, $\pi_X(R_1 \uplus R_2) = \pi_X(R_1) \uplus \pi_X(R_2)$ for all bags, so $\text{ans}(t) = \text{ans}(l) \uplus \text{ans}(r)$. For COUNT$(*)$, the count of a disjoint union is the sum of the counts. SUM adds the non-NULL measure values of its input and returns NULL when there are none. If both parts contain a non-NULL value, the target's sum is the sum of the two parts' sums. If exactly one part does, the target's sum equals that part's sum, and the other part returns NULL, which counts as zero. If neither part does, the target returns NULL and so do both parts. For grouped aggregates, selection keeps the value of g, so $G_v(t) = G_v(l) \uplus G_v(r)$ for every value v, including NULL. A key occurs in $\text{ans}(t)$ if and only if it occurs in $\text{ans}(l)$ or $\text{ans}(r)$, and the argument above applies to each key, with a key missing from one part treated as an empty group of that part. The identities hold in exact arithmetic; DuckDB's floating-point sums agree within the tolerances of Section 4.3.

Rule R2. The queries q_d and q_s read T with the same predicate ϕ, and $X_d \subset X_s$. For every bag R, $\pi_{X_d}(\pi_{X_s}(R)) = \pi_{X_d}(R)$, because both sides map each tuple of R to the same restriction and keep its multiplicity. With $R = \sigma_\phi(T)$, $\text{ans}(d) = \pi_{X_d}(\text{ans}(s))$.

Rule R3. The predicates of q_d and q_s use the same kind and column c, ϕ_d is a sub-interval or a proper category subset of ϕ_s, $X_d \subseteq X_s$, and $c \in X_s$. Every tuple that satisfies ϕ_d satisfies ϕ_s, so $\sigma_{\phi_d}(T) = \sigma_{\phi_d}(\sigma_{\phi_s}(T))$. Because ϕ_d reads only c, which π_{X_s} keeps, $\sigma_{\phi_d}(\pi_{X_s}(R)) = \pi_{X_s}(\sigma_{\phi_d}(R))$ for every bag R. Therefore

$$\pi_{X_d}\big(\sigma_{\phi_d}(\text{ans}(s))\big) = \pi_{X_d}\big(\pi_{X_s}(\sigma_{\phi_d}(\sigma_{\phi_s}(T)))\big) = \pi_{X_d}\big(\sigma_{\phi_d}(T)\big) = \text{ans}(d),$$

where the middle step uses the identity of R2.

Rule R4. The ungrouped aggregate q_d has the table, aggregate, measure, and predicate ϕ of the grouped q_s, so $\text{ans}(d)$ aggregates $\sigma_\phi(T) = \biguplus_v G_v(s)$. Its COUNT$(*)$ is the sum of the group counts, which is zero when $\text{ans}(s)$ has no rows. Its SUM is the sum of the non-NULL group sums, and it is NULL exactly when every group sum is NULL or $\text{ans}(s)$ has no rows.

Rules R5 and R6. The predicate $g \in S$ holds for a tuple exactly when its value of g is a non-NULL member of S. If q_s has no predicate, then $\sigma_{g \in S}(T) = \biguplus_{v \in S} G_v(s)$, where $G_v(s)$ is empty when v does not occur. If the predicate of q_s is a category set P on g and $S \subset P$, every tuple with $g \in S$ satisfies P, so the same identity holds. For R5, the ungrouped q_d aggregates this union, so its value is the sum of the values of the groups of $\text{ans}(s)$ with keys in S, with the NULL rule of R4. For R6, q_d groups $\sigma_{g \in S}(T)$ by g, and each of its groups equals $G_v(s)$ for the same $v \in S$, so $\text{ans}(d)$ consists of the rows of $\text{ans}(s)$ whose key lies in S. □

A buyer who holds $\text{ans}(l)$ and $\text{ans}(r)$, or $\text{ans}(s)$, computes $\text{ans}(t)$, or $\text{ans}(d)$, without querying the database. A price with $p_t > p_l + p_r$ or $p_d > p_s$ would therefore sell the same information for less, so every row that the compiler emits states a necessary condition for arbitrage freedom.

2.2 Chain audit

Soundness of a chain. Let q_t have the half-open interval $[a, b)$ on column c, and let $q_{l_1}, \ldots, q_{l_k}$ share its surface with intervals $[x_{i-1}, x_i)$ for $a = x_0 < x_1 < \cdots < x_k = b$. Every non-NULL value of c in $[a, b)$ lies in exactly one of the parts, and a NULL value lies in none, so $\sigma_{[a,b)}(T) = \biguplus_{i=1}^{k} \sigma_{[x_{i-1}, x_i)}(T)$. Projection, grouping, COUNT, and SUM distribute over this disjoint union as in the proof of R1, applied $k - 1$ times, so the parts determine $\text{ans}(t)$, and the row $p_t - \sum_{i=1}^{k} p_{l_i} \le 0$ is necessary for arbitrage freedom.

Cheapest chain. Fix the surface of q_t and form the directed graph whose nodes are a, b, and the endpoints of the surface's other intervals inside $[a, b)$, with one edge from x to y of weight p_j for each such interval $[x, y)$ of an asset q_j. Every edge goes from a smaller to a larger endpoint, so the graph is acyclic, and the chains of parts that tile $[a, b)$ correspond one to one to the directed paths from a to b, with chain cost equal to path weight. The audit computes the minimum path weight by dynamic programming over the endpoints in increasing order, which is exact for any real weights, including slightly negative prices that a released vector may hold within the tolerance. The audit flags the cheapest chain, with k parts, when p_t exceeds

its cost by more than $k\varepsilon$. Lazy enforcement appends each flagged chain as a row with $k+1$ coefficients through the snapshot's append path, which leaves the existing rows unchanged, and gates the vector again. Proposition 6.7 then applies with $\alpha = \sqrt{k_{\max}+1}$ for the longest appended chain; in the extension campaign, chains have at most five parts, so $\alpha \le \sqrt{6}$.

2.3 Price floors

Let the list prices ℓ satisfy every row and let $f \ge 0$. For a decomposition row, $\ell_t \le \ell_l + \ell_r$ gives $\max(\ell_t, f) \le \max(\ell_l + \ell_r, f) \le \max(\ell_l, f) + \max(\ell_r, f)$: the first step holds because $x \mapsto \max(x, f)$ is nondecreasing, and the second because $\ell_l, \ell_r \ge 0$. For a derivation row, $\ell_d \le \ell_s$ gives $\max(\ell_d, f) \le \max(\ell_s, f)$, and every floored price is nonnegative. The floored vector therefore satisfies every row, every target list price becomes at least f, and Corollary 6.4 bounds each leftover saving by ε/f relative to list price.

2.4 Encoded arbitrage safety

Assumption 6.1 (trusted boundary). *(i) The compiler maps each request to its intended asset and emits only relations that satisfy rules R1 to R6. (ii) The snapshot store returns an immutable $(\mathbf{A}_e, \mathbf{b}_e)$ for epoch e. (iii) The verifier evaluates every row on finite binary64 values. (iv) Commit compares the token with the current state and performs the release in one atomic step. (v) SHA-256 is collision resistant, and the HMAC secret is known only to the service.*

Lemma 6.2 (post-check implication). *If the verifier accepts a finite vector $\mathbf{p}$ for $(\mathbf{A}_e, \mathbf{b}_e)$, then $\mathbf{a}_j^\top \mathbf{p} - b_j \le \varepsilon$ for every row j.*

Proof. The verifier recomputes all m residuals and accepts only if $\rho_e(\mathbf{p}) \le \varepsilon$ (Equation 2), so no residual exceeds ε. □

Theorem 6.3 (no encoded arbitrage above tolerance). *Under Assumption 6.1, suppose the commit releases an answer with vector $\mathbf{p}$ and a token for epoch e. Then $p_t - p_l - p_r \le \varepsilon$ for every decomposition $(t, l, r) \in \mathcal{R}_e$, and $p_d - p_s \le \varepsilon$ for every derivation $(d, s) \in \mathcal{R}_e$.*

Proof. A release requires a commit that re-verifies $\mathbf{p}$ against the current snapshot, and the token's snapshot digest identifies $(\mathbf{A}_e, \mathbf{b}_e)$. By Lemma 6.2, every row residual is at most ε. The compiler emits for (t, l, r) a row with coefficient $+1$ on p_t, -1 on p_l and p_r, and bound 0, and for (d, s) a row with $+1$ on p_d, -1 on p_s, and bound 0. Each residual therefore equals the saving of its relation. A chain row appended by the audit has $+1$ on its target and -1 on each part, so its saving is also at most ε. □

2.5 State and request binding

Lemma 6.5 (stale-token rejection). *Under Assumption 6.1, a token issued for epoch e cannot authorize a release after the state advances to $e' \ne e$.*

Proof. Epoch equality is a required commit predicate, so it is false when $e' \ne e$ and the commit retains the answer. Atomicity prevents a state change between a successful comparison and the release. □

Lemma 6.6 (single bound release). *Under Assumption 6.1, a token authorizes at most one release, and only for the request, asset, quoted price, price vector, and answer bytes it names.*

Proof. Every request field, the price-vector digest, the buffer identifier, and the answer digest are inside the HMAC, so changing any of them without the secret invalidates the token. Commit compares each field with the presented request and buffer, and it compares the quote with the vector coordinate of the requested asset. The release transaction inserts the token, the request, and the buffer under uniqueness constraints, so a second commit of any of them aborts. □

2.6 Convergence of active-row repair

For a nonempty polyhedron $\mathcal{C}_e$, Hoffman's bound [6] gives a constant μ that depends only on the rows of $\mathbf{A}_e$, such that

$$\operatorname{dist}(\mathbf{p}, \mathcal{C}_e) \le \mu \max_j \frac{\delta_j(\mathbf{p})^+}{\|\mathbf{a}_j\|_2} \quad \text{for every } \mathbf{p}.$$

Every halfspace $H_j = \{\mathbf{p} : \mathbf{a}_j^\top \mathbf{p} \le b_j\}$ contains $\mathcal{C}_e$, and $\operatorname{dist}(\mathbf{p}, H_j) = \delta_j(\mathbf{p})^+ / \|\mathbf{a}_j\|_2$, so $\operatorname{dist}(\mathbf{p}, \mathcal{C}_e) \ge \max_j \operatorname{dist}(\mathbf{p}, H_j)$ and therefore $\mu \ge 1$ whenever some $\mathbf{p}$ lies outside $\mathcal{C}_e$.

Proposition 6.7 (convergence of active-row repair). *Assume that $\mathcal{C}_e$ is nonempty and that arithmetic is exact, and let $\gamma = \eta(2-\eta)\,\omega^2/(\alpha^2\mu^2)$. For any step $0 < \eta \le 1$ and any active-row budget $B \ge 1$, every sweep that starts at $\mathbf{p}^{(k)}$ with $\rho_e(\mathbf{p}^{(k)}) > \varepsilon$ ends at a vector with $\operatorname{dist}(\mathbf{p}^{(k+1)}, \mathcal{C}_e)^2 \le (1-\gamma)\operatorname{dist}(\mathbf{p}^{(k)}, \mathcal{C}_e)^2$. Repair therefore reaches $\rho_e \le \varepsilon$ within $\lceil (2/\gamma)\ln^+(\alpha \operatorname{dist}(\mathbf{p}^{(0)}, \mathcal{C}_e)/\varepsilon) \rceil$ sweeps, where $\ln^+ x = \max(\ln x, 0)$.*

Proof. Step 1: one update. Let $\delta_j = \delta_j(\mathbf{p}) > 0$ and $\mathbf{p}' = \mathbf{p} - \eta\,\delta_j\,\mathbf{a}_j/\|\mathbf{a}_j\|_2^2$. For every $\mathbf{c} \in \mathcal{C}_e$, $\mathbf{a}_j^\top \mathbf{c} \le b_j$, so $\mathbf{a}_j^\top(\mathbf{p} - \mathbf{c}) \ge \delta_j$. Expanding the square,

$$\|\mathbf{p}' - \mathbf{c}\|_2^2 = \|\mathbf{p} - \mathbf{c}\|_2^2 - \frac{2\eta\,\delta_j\,\mathbf{a}_j^\top(\mathbf{p} - \mathbf{c})}{\|\mathbf{a}_j\|_2^2} + \frac{\eta^2\delta_j^2}{\|\mathbf{a}_j\|_2^2} \le \|\mathbf{p} - \mathbf{c}\|_2^2 - \frac{\eta(2-\eta)\,\delta_j^2}{\|\mathbf{a}_j\|_2^2}.$$

With $\mathbf{c}$ the projection of $\mathbf{p}$ onto $\mathcal{C}_e$, this gives $\operatorname{dist}(\mathbf{p}', \mathcal{C}_e)^2 \le \operatorname{dist}(\mathbf{p}, \mathcal{C}_e)^2 - \eta(2-\eta)\,\delta_j^2/\|\mathbf{a}_j\|_2^2$. In particular, no update increases the distance, because $\eta(2-\eta) > 0$.

Step 2: the rest of a sweep. Repair updates a visited row only while its current residual exceeds $\varepsilon > 0$, and it leaves $\mathbf{p}$ unchanged otherwise. By Step 1, the distance to $\mathcal{C}_e$ never increases during a sweep.

Step 3: the first update of a sweep. If $\rho_e(\mathbf{p}^{(k)}) > \varepsilon$, the violated set is nonempty, and the first visited row j^* has the largest residual $\delta = \max_j \delta_j(\mathbf{p}^{(k)}) = \rho_e(\mathbf{p}^{(k)}) > \varepsilon$. No update precedes it in the sweep, so its residual is still δ when repair visits it, and repair applies the update. Let $\mathbf{p}_1^{(k)}$ be the result. By Step 1 and $\|\mathbf{a}_{j^*}\|_2 \le \alpha$,

$$\operatorname{dist}(\mathbf{p}_1^{(k)}, \mathcal{C}_e)^2 \le \operatorname{dist}(\mathbf{p}^{(k)}, \mathcal{C}_e)^2 - \frac{\eta(2-\eta)\,\delta^2}{\alpha^2}.$$

Step 4: Hoffman's bound. Every row has $\delta_j(\mathbf{p}^{(k)})^+ \le \delta$ and $\|\mathbf{a}_j\|_2 \ge \omega$, so $\max_j \delta_j(\mathbf{p}^{(k)})^+/\|\mathbf{a}_j\|_2 \le \delta/\omega$ and Hoffman's bound gives $\delta \ge \omega \operatorname{dist}(\mathbf{p}^{(k)}, \mathcal{C}_e)/\mu$. Substituting into Step 3 and applying Step 2 to the remaining updates of the sweep,

$$\operatorname{dist}(\mathbf{p}^{(k+1)}, \mathcal{C}_e)^2 \le \operatorname{dist}(\mathbf{p}_1^{(k)}, \mathcal{C}_e)^2 \le \Big(1 - \frac{\eta(2-\eta)\,\omega^2}{\alpha^2\mu^2}\Big)\operatorname{dist}(\mathbf{p}^{(k)}, \mathcal{C}_e)^2 = (1-\gamma)\operatorname{dist}(\mathbf{p}^{(k)}, \mathcal{C}_e)^2.$$

Step 5: residuals and distance. For every $\mathbf{p}$, let $\mathbf{c}$ be its projection onto $\mathcal{C}_e$. Then $\delta_j(\mathbf{p}) \le \mathbf{a}_j^\top \mathbf{p} - \mathbf{a}_j^\top \mathbf{c} \le \|\mathbf{a}_j\|_2\,\|\mathbf{p} - \mathbf{c}\|_2 \le \alpha \operatorname{dist}(\mathbf{p}, \mathcal{C}_e)$ for every row, so $\rho_e(\mathbf{p}) \le \alpha \operatorname{dist}(\mathbf{p}, \mathcal{C}_e)$.

Step 6: sweep count. The constant γ lies in $(0, 1]$ because $\mu \ge 1$, $\omega \le \alpha$, and $\eta(2-\eta) \le 1$. If repair has not stopped after k sweeps, each of them started with $\rho_e > \varepsilon$, so Step 4 gives $\operatorname{dist}(\mathbf{p}^{(k)}, \mathcal{C}_e)^2 \le (1-\gamma)^k \operatorname{dist}(\mathbf{p}^{(0)}, \mathcal{C}_e)^2 \le e^{-\gamma k} \operatorname{dist}(\mathbf{p}^{(0)}, \mathcal{C}_e)^2$. By Step 5, $\rho_e(\mathbf{p}^{(k)}) \le \alpha\, e^{-\gamma k/2} \operatorname{dist}(\mathbf{p}^{(0)}, \mathcal{C}_e)$, which is at most ε once $k \ge (2/\gamma)\ln(\alpha \operatorname{dist}(\mathbf{p}^{(0)}, \mathcal{C}_e)/\varepsilon)$. If $\alpha \operatorname{dist}(\mathbf{p}^{(0)}, \mathcal{C}_e) \le \varepsilon$, Step 5 already gives $\rho_e(\mathbf{p}^{(0)}) \le \varepsilon$ and no sweep runs. A sweep with $\rho_e > \varepsilon$ always changes the vector (Step 3), so the stopping rule for an unchanged sweep never ends repair early. □

Remarks. The argument uses only the first update of each sweep, so it holds for every budget $B \ge 1$ and every order of the remaining visited rows; it also holds for every step $0 < \eta < 2$. The classical relaxation method of Agmon [1] projects onto a farthest halfspace and converges at a geometric rate; a sweep of active-row repair starts with the largest residual instead of the largest distance, which costs at most the factor ω/α in Step 3. For the compiled rows of the paper, every row has one to three coefficients of magnitude one, so $\omega = 1$ and $\alpha = \sqrt{3}$, and $\mathbf{p} = \mathbf{0}$ satisfies every decomposition, derivation, nonnegativity, cap, and chain row, so $\mathcal{C}_e$ is nonempty. The proposition does not bound μ, which can be large, and it does not compare the repaired vector with the Euclidean projection of Equation 4. In binary64 arithmetic, the verifier, not this proposition, decides acceptance.

2.7 Bracketing the exact projection

Let $f(\mathbf{p}) = \frac{1}{2}\|\mathbf{p} - \mathbf{p}^{(0)}\|_2^2$ and let $\mathbf{p}^\star$ minimize f over $\mathcal{C}_e$ (Equation 4). For multipliers $\boldsymbol{\lambda} \geq 0$, the Lagrange dual function is

$$g(\boldsymbol{\lambda}) = \min_{\mathbf{p}} f(\mathbf{p}) + \boldsymbol{\lambda}^\top(\mathbf{A}_e\mathbf{p} - \mathbf{b}_e) = \boldsymbol{\lambda}^\top(\mathbf{A}_e\mathbf{p}^{(0)} - \mathbf{b}_e) - \tfrac{1}{2}\|\mathbf{A}_e^\top\boldsymbol{\lambda}\|_2^2,$$

attained at $\mathbf{p} = \mathbf{p}^{(0)} - \mathbf{A}_e^\top\boldsymbol{\lambda}$, and weak duality gives $g(\boldsymbol{\lambda}) \leq f(\mathbf{p}^\star)$. Hence $L = \sqrt{2g(\boldsymbol{\lambda})} \leq \|\mathbf{p}^\star - \mathbf{p}^{(0)}\|_2$ whenever $g(\boldsymbol{\lambda}) > 0$. For every $\mathbf{q} \in \mathcal{C}_e$, $U = \|\mathbf{q} - \mathbf{p}^{(0)}\|_2 \geq \|\mathbf{p}^\star - \mathbf{p}^{(0)}\|_2$. Because f is 1-strongly convex and $\mathbf{p}^\star$ minimizes it over the convex set $\mathcal{C}_e$, $f(\mathbf{q}) - f(\mathbf{p}^\star) \geq \frac{1}{2}\|\mathbf{q} - \mathbf{p}^\star\|_2^2$, so $\|\mathbf{q} - \mathbf{p}^\star\|_2 \leq \Delta = \sqrt{2(f(\mathbf{q}) - g(\boldsymbol{\lambda}))}$. The output $\mathbf{x}$ of a method then satisfies $\|\mathbf{x} - \mathbf{p}^\star\|_2 \in [\|\mathbf{x} - \mathbf{q}\|_2 - \Delta, \|\mathbf{x} - \mathbf{q}\|_2 + \Delta]$, and its L2 change ratio $\|\mathbf{x} - \mathbf{p}^{(0)}\|_2 / \|\mathbf{p}^\star - \mathbf{p}^{(0)}\|_2$ lies in $[\|\mathbf{x} - \mathbf{p}^{(0)}\|_2 / U, \|\mathbf{x} - \mathbf{p}^{(0)}\|_2 / L]$.

Choosing $\boldsymbol{\lambda}$. Candidates are the dual vectors that Clarabel returns, and multipliers recovered from the best near-feasible output $\mathbf{x}$: a least-squares solution of the stationarity condition $\mathbf{p}^{(0)} - \mathbf{x} = \mathbf{A}_e^\top\boldsymbol{\lambda}$ restricted to the rows with residual at least -10^{-9}, computed with LSMR and clipped at zero. Every $\boldsymbol{\lambda} \geq 0$ gives a valid bound, so a poor recovery only weakens L. When the best candidate leaves a relative gap above 10^{-8} to the best objective, Hildreth's dual coordinate ascent [5], warm-started from it and capped at 3 s, raises $g(\boldsymbol{\lambda})$; each coordinate step increases g and keeps $\boldsymbol{\lambda} \geq 0$.

Choosing $\mathbf{q}$. Candidates are the outputs whose computed residuals are all at most zero, the point $\ell + \text{median}(\ell)$, which satisfies every decomposition row with slack $\text{median}(\ell)$ and every derivation and nonnegativity row strictly, and blends of each other output toward each strictly feasible candidate, with the smallest step that makes every computed residual nonpositive. Both bounds are evaluated in binary64 arithmetic, so the narrowest brackets of Section 5.5 are at the level of rounding error.

3 Model Hyperparameters

All three model families come from scikit-learn 1.6.1, and their hyperparameters are fixed in the code; no search was run. Each model is a pipeline that standardizes five numeric query features (output count, category count, interval width, SQL length, and log estimated rows) and one-hot encodes five categorical ones (table, query shape, predicate kind, aggregate, and the generator's template family). The target is the seller's list price. The median predictor is a `DummyRegressor` that returns the median list price of its training folds. Histogram gradient boosting uses a learning rate of 0.05, 300 iterations, 31 leaves, and an L2 penalty of 10^{-3}. The multilayer perceptron has two hidden layers of 32 and 16 tanh units and trains with Adam at a learning rate of 5×10^{-4}, a batch size of 128, and an L2 penalty of 10^{-3}, with early stopping on 15% of the training data, a patience of 40 iterations, and at most 1,000 iterations. The code records two earlier choices for the perceptron: tanh replaced ReLU after one held-out fold on UCI Adult produced prices two orders of magnitude outside the label range, and Adam with early stopping replaced L-BFGS because L-BFGS scaled poorly at 4,096 assets. Training uses five-fold cross-validation grouped by SQL template, and seeds 11, 23, 37, 51, and 73 give 45 proposals. The extension campaign retrains the same three families, with the same hyperparameters, on the workload with grouped assets and on each floored policy.

4 Base-Workload Results Summarized in Section 9

4.1 Buyer-history snapshots

Section 9.1 prices the next purchase of 10 returning buyers per history size, dataset, and proposal (the list prices and one gradient-boosting proposal). Owning 16, 64, or 256 answers makes 26 to 969 further answers derivable, and 41 to 640 decompositions are rewritten over missing parts; owned and derivable answers leave the buyer's vector. All 360 history snapshots are released without fallback, the residual variant within 115 sweeps and the list-cap variant within 377. Table 1 gives every cell.

Table 1: Buyer-history snapshots at tolerance 10^{-12}, 120 per row (10 buyers per dataset, proposal, and variant), all released without fallback. Derivable: answers computable from owned ones. Remaining: answers still priced. Rewritten: decompositions rewritten over missing parts. Sweeps: maximum for the residual / list-cap variants. p50: median gate time of the residual variant in ms.

Owned	Derivable	Remaining	Rewritten	Sweeps	p50
16	26–104	3,976–4,054	41–173	115/359	95.1
64	151–330	3,702–3,881	207–412	107/377	177
256	677–969	2,871–3,163	444–640	114/307	173

4.2 Seller price shapes

We vary the seller's policy instead of a buyer model. Besides the linear policy of Equation 6, a concave policy applies a square root to the extent, and a tiered policy charges each asset the largest extent in its tier. We requested eight equal-count tiers, and ties leave seven, seven, and five distinct tiers on the three datasets. We rescale both alternatives to the linear mean. The linear and concave policies violate no row of their own snapshots. The tiered policy violates 539, 330, and 122 rows on NYC Taxi, TPC-H SF1, and UCI Adult, with residuals up to 1.08, because rounding up to tier levels breaks the additivity that decomposition rows require. The tiered-shape attacks therefore start from the gate-repaired tiered policy. We retrain all three model families on each shape, which gives pooled R^2 from -0.59 for the median predictor to 0.9999 for gradient boosting. All 135 retrained proposals and all 540 shape attacks are released without fallback within 73 sweeps. The release decision therefore does not depend on the seller policy being feasible by construction.

4.3 Scale and snapshot cost

Figure 2 of the paper plots the gate and the baselines against snapshot size on attack candidates that violate 1 to 51 rows (median 7). Arbiter's median grows from 0.19 to 0.21 ms at 128 assets to 0.80 to 0.85 ms at 4,096 assets, with a p95 at or below 1.02 ms. Check only takes 0.04 to 0.41 ms, so repair adds 0.15 to 0.44 ms to verification at the median. Full cyclic repair needs up to 18 sweeps over all rows and reaches 56 to 110 ms at 4,096 assets, with a p95 up to 958 ms; its latency follows the number of candidates that need more than one sweep, so it is not monotone in size. OSQP with $\epsilon_{\text{abs}} = 10^{-12}$ verifies every candidate and takes 117 to 156 ms at 4,096 assets. With the audited stopping rule, $\epsilon_{\text{abs}} = 10^{-9}$, OSQP takes 82 to 123 ms, but its output passes verification at 10^{-12} in only 75% to 100% of the trials in a cell. At 128 assets, the paired speedups are 5.3 to 5.9 over full cyclic repair and 8.8 to 9.9 over OSQP; at 4,096 assets, the paper gives them with their intervals, which are wide against full cyclic repair because its latency per candidate follows its sweep count. Building a 4,096-asset snapshot takes 228 to 244 ms at the median; its numeric CSR arrays occupy 454 to 481 KB, and diagnostic row metadata brings the snapshot to 4.9 to 5.1 MB. In the audited campaign, appending relations was 2.1 times faster than a rebuild and gave an identical row multiset in 90 of 90 trials.

4.4 Model proposals in the revision campaign

Table 2 gives the revision campaign's comparison on the 45 model proposals, which the paper replaces with the extension campaign's comparison (Section 5.5 here). Arbiter's median is 64.9 to 84.5 ms. The paired speedups over full cyclic repair are 11.2 [10.5, 17.4] on NYC Taxi, 17.0 [9.54, 31.2] on TPC-H SF1, and 14.4 [8.41, 28.5] on UCI Adult. OSQP at 10^{-12} takes 0.45 to 9.2 s at the median and stops at its 20,000-iteration limit in 54 of 135 trials, and the verifier passed all 135 results. OSQP with the audited stopping rule passes verification in only 30 of 135 trials. The paired speedups over OSQP at 10^{-12} are 7.06 [3.74, 7.17], 111 [3.97, 296], and 54.7 [2.22, 248]. In the fallback configuration, OSQP took up to 12.6 s on these proposals. The L1 norm of Arbiter's price change is a median 1.06 times that of the OSQP solution of Equation 4 (range 1.02 to 1.11 over the 45 proposals) and 1.00 times that of full cyclic repair.

Table 2: Repair latency on the 45 model proposals at tolerance 10^{-12}, median / p95 in ms (15 proposals per dataset, 3 timed repetitions after one warm-up, counterbalanced order). OSQP rows give ϵ_{abs}. All outputs of Arbiter, full cyclic repair, and OSQP at 10^{-12} passed verification; OSQP at 10^{-9} passed in 33%, 0%, and 33% of trials; check only refuses all.

Method	NYC Taxi	TPC-H SF1	UCI Adult
Arbiter	64.9 / 117	84.5 / 109	72.4 / 103
Full cyclic	1,102 / 1,421	981 / 1,621	1,193 / 1,501
OSQP, 10^{-12}	453 / 11,477	9,212 / 9,488	4,607 / 12,595
OSQP, 10^{-9}	248 / 948	256 / 620	161 / 2,221
Check only	0.40 / 0.48	0.39 / 0.67	0.41 / 0.44

Table 3: Protocol behavior. Top: campaigns rerun at 10^{-12}; a case passes when the gate releases exactly when the specification requires. Bottom: median control-path latency in ms from the audited runs on 2-vCPU hosts at 10^{-7}, excluding query execution.

Campaign	Cases	Passed
Token and commit faults	22 per dataset	66/66
Stale-epoch races	2 schedules × 100	600/600
Request-binding faults	21 per dataset	63/63
Cross-service races	2 schedules × 50	300/300
Delivery authorization	2 per dataset	6/6
Control path		**p50 (ms)**
Gate and commit, in memory		6.77–7.38
Request-bound durable commit		17.0–17.5
Single-stage path, scalar to medium		20.5–28.4
Single-stage path, 3.47 MB answer		761

4.5 Protocol campaigns

Table 3 reports the protocol campaigns, rerun at 10^{-12}; a case passes when the gate releases exactly when the specification requires. The 66 token and commit faults, 22 per dataset, cover tampering, stale epochs, replays, non-finite values, solver and fallback failures, and snapshot mutation after verification. The 600 stale-epoch race trials run two schedules of 100 trials per dataset: in the simultaneous schedule, 45 trials released and every outcome matched its epoch order, and in the update-before-commit schedule, none released. The 63 request-binding faults, 21 per dataset, cover swapped fields and buffers, expired, replayed, and cross-request tokens, and service restarts. The 300 cross-service races run two schedules of 50 trials per dataset, and the six delivery checks are two per dataset. On the 2-vCPU hosts of the audited campaign, at 10^{-7} and excluding query execution, the in-memory gate and commit took 6.77 to 7.38 ms at the median, the request-bound durable commit 17.0 to 17.5 ms at the median and 18.7 to 20.7 ms at the p95, and the single-stage path 20.5 to 28.4 ms for scalar to medium answers and 761 ms for a 3.47 MB answer. Serializing and digesting that answer took 664 ms at the median.

4.6 Active-row budget under dense violations

We gate 18 dense candidates per setting, with 8 to 199 violated rows, under budgets of 1, 4, 8, 16, 32, and 64 rows per sweep (Table 4). At 10^{-12} with 80 sweeps, budgets of 1, 4, and 8 leave 3, 2, and 1 candidates unverified by the active path, and budgets of 16 or more leave none. With 400 sweeps, only the budget of 1 leaves a candidate unverified. At 10^{-7} with 80 sweeps, budgets of 1 and 4 leave 3 and 1, as in the audited campaign. The fail-safe tier verified every candidate that the active path left unverified: full cyclic repair resolved five of the six at $(10^{-12}, 80)$, and OSQP resolved the sixth. The budget therefore changes how often the gate falls back, never what it accepts.

Table 4: Active-row budget under dense violations: candidates left unverified by the active path, out of 18 per setting (8 to 199 violated rows per candidate). The fail-safe tier verified every candidate that the active path left unverified. The last column counts candidates whose violated set exceeded the budget at 10^{-12} with 80 sweeps.

Budget B	10^{-12}, 80	10^{-12}, 400	10^{-7}, 80	Budget binding
1	3 / 18	1 / 18	3 / 18	18
4	2 / 18	0 / 18	1 / 18	18
8	1 / 18	0 / 18	0 / 18	17
16	0 / 18	0 / 18	0 / 18	12
32	0 / 18	0 / 18	0 / 18	9
64	0 / 18	0 / 18	0 / 18	9

4.7 Fallback kernel

On the six candidates that needed fallback at $(10^{-12}, 80)$, the violated-only kernel is 77 to 142 times faster than a full scan when both run up to 80 sweeps, with medians of 1.9 to 12.6 ms against 165 to 1,148 ms, and both verify all six. With 20 sweeps, the violated-only kernel verifies only 3 of the 6 and the full scan 5 of the 6. A 20-sweep fallback budget is therefore too small at 10^{-12}, so we set the budget to 80. In the fail-safe tier, OSQP verified the cases that the shorter budget missed.

4.8 Sweep budget and recomputation

The largest sweep counts were 63 for natural proposals at 10^{-12}, 136 for declaration-coverage gates, 73 for shape proposals, 115 for residual history snapshots, and 377 for list-cap snapshots. A budget of 80, sized for 10^{-7}, would have sent 80.6% of residual history gates and 41.1% of list-cap gates to fallback. Recomputing a 4,096-asset seller policy takes 1.84 to 2.55 ms at the median over 200 repetitions, depending on the shape, and verifying a supplied vector takes 0.43 to 0.47 ms. Recomputation cannot replace verification: the tiered policy recomputes to an infeasible vector, and the boundary cannot recompute a price that a contract or an external service supplies.

4.9 Tolerance and price scale

A fixed absolute tolerance changes its meaning when prices are rescaled: if all prices are multiplied by 100, the same ε becomes 100 times stricter relative to every price. The floor study of Section 5.4 varies the smallest list price directly.

5 The Extension Campaign

5.1 Environment and protocol

The extension campaign ran in one Colab session on one host with an AMD EPYC 7B12, eight logical CPUs, and 54.8 GB of memory, with Python 3.13.16, NumPy 2.1.3, pandas 2.2.3, SciPy 1.16.3, scikit-learn 1.6.1, OSQP 1.0.4, Clarabel 0.11.1 [4], DuckDB 1.3.2, and SQLGlot 27.0.0. It ran 28 checkpointed parts in 3,918 s of stage time and passed all 45 of its quality checks, 40 of them fatal integrity checks, and the audited package was byte-identical after the campaign. Before any experiment, it regenerated the audited 4,096-asset workloads, their 28,497 relations, and their snapshot digests exactly. The projection study times every method over 3 repetitions after one warm-up, in rotated method order. Scale-out cells time Arbiter and check only over 10 repetitions after 3 warm-ups, and full cyclic repair, OSQP, and Clarabel over 3 repetitions after one. Speedups are paired over units, with 95% percentile-bootstrap intervals over 10,000 resamples, as in the paper. We compare these timings only with each other.

5.2 GROUP BY extension

Workload. A deterministic generator derives grouped aggregates from the audited catalog statistics in groups that close under the new rules: a grouped answer over all rows with its ungrouped total; a grouped interval partition, a target and two parts, with their three ungrouped totals; and a grouped category partition on the grouping column with its three ungrouped totals. Groups are taken from the tables in turn, with up to three grouping columns per table, and assets that already exist are not duplicated. This gives 704 new assets (164 on NYC Taxi, 352 on TPC-H SF1, and 188 on UCI Adult), 504 of them grouped, and 1,424 new relations: 170 grouped partition unions, 74 partition unions among new ungrouped assets, 504 roll-ups, 338 slices, and 338 group filters. A grouped answer costs 12% more than its ungrouped total under Equation 6, and the base workload's normalization reproduces every base price exactly.

Validation. Table 5 summarizes the compiler checks. The extended parser passes all 32 fixed strings of its self-test: six supported shapes, 25 fail-closed shapes, and a renderer round trip. The fail-closed shapes are two grouping columns, `HAVING`, `AVG`, `ORDER BY`, `LIMIT`, `DISTINCT`, `ROLLUP`, `CUBE`, `GROUPING SETS`, `GROUP BY ALL`, a positional key, a selected column other than the key, an alias on the key, the aggregate before the key, two aggregates, `COUNT(DISTINCT)`, `SUM(DISTINCT)`, an expression inside `SUM`, `FILTER`, a predicate outside the supported forms, a schema-qualified table, a subquery, a common table expression, a join, and a set operation. The base assets parse exactly as in the audited compile, and every new asset parses to its construction descriptor. An independent pairwise oracle, which compares every grouped asset with every aggregate asset of its dataset from construction descriptors, matches the compiler's 1,424 relations exactly. All 180 executed checks pass in DuckDB on the real data. Each check composes in SQL the answers that a relation names, as a buyer who holds them would, and compares the result with the priced query's answer group by group, counts exactly and sums within a relative 10^{-10} and an absolute 10^{-8}. The 17 audited unsupported queries of the form `SELECT g, COUNT(*) AS n FROM t GROUP BY g` now compile to grouped assets, and the other 122 unsupported queries are still rejected.

Release. The three families, retrained over the extended workload, reach pooled out-of-fold R^2 from -0.50 to -0.45 for the median predictor, 0.990 to 0.993 for gradient boosting, and 0.9988 to 0.9996 for the perceptron. Their 45 proposals violate 0 to 3,494 rows (median predictor), 2,063 to 4,209 rows (gradient boosting), and 3,287 to 4,809 rows (perceptron) before the gate, up to 184 of them from the new families. The gate releases all 45 within 56 sweeps and without fallback, with median gate times of 88.3, 61.4, and 63.4 ms on NYC Taxi, TPC-H SF1, and UCI Adult. The 888 exact per-row attacks on the new families, with budgets of 10%, 25%, and 50% and up to 20 rows per family, dataset, and budget, violate 1 to 21 rows. The gate releases all of them without fallback, at a median of 0.39 ms and a p95 of 0.48 ms; grouped partition unions need up to 50 sweeps, and the other families at most two. Every released vector passed independent verification.

Table 5: GROUP BY extension in the extension campaign, summed over the three datasets: compiled and oracle relations per rule family, the smallest per-dataset precision and recall, executed checks passed in DuckDB, exact per-row attacks released at tolerance 10^{-12} with no fallback and no unverified release, and the largest sweep count of those releases.

Family	Compiled	Oracle	Prec.	Rec.	Checks	Attacks	Sweeps
Grouped partition union (R1)	170	170	1.000	1.000	36/36	180/180	50
Partition union, new assets (R1)	74	74	1.000	1.000	36/36	168/168	2
Roll-up (R4)	504	504	1.000	1.000	36/36	180/180	1
Slice (R5)	338	338	1.000	1.000	36/36	180/180	1
Group filter (R6)	338	338	1.000	1.000	36/36	180/180	1

5.3 Chain audit

Targets. For every interval surface, candidate targets span five, six, or eight adjacent catalog boundary intervals. A candidate is admitted when it is not an asset, no two assets of its surface tile it, it does not complete a two-part split of an admitted target, and a chain of three or more existing intervals tiles it, taking surfaces in turn up to 60 targets per dataset. R1 emits no row for any of the 180 targets, list prices that include them verify, and the audit flags nothing on list prices.

Results. Table 6 summarizes the audit. Each attack raises one target's price within the box of Equation 3, with budgets of 10%, 25%, and 50%. The gate releases all 540 attacked vectors, because no declared row covers the chain, and the audit detects all 540. The cheapest flagged chain has three to five parts. Lazy enforcement needed two rounds for 76 attacks, three for 410, four for 43, and five for 11, so three at the median, and the audit flags nothing after enforcement; every final release passed verification. The first gate takes 0.34 ms at the median, and the whole enforcement takes 222 ms at the median and 468 ms at most. One audit pass over all 1,331 to 1,389 interval assets of a dataset takes 18 to 23 ms at the median.

Audit mode. For 15 proposals per dataset, the gate releases a vector against a declaration of decompositions only (D) or of projections and filters only (P+F), and the audit then checks the release against the complete snapshot. It flags 45 of 45 releases under D and 30 of 45 under P+F, at 0.20 to 0.22 ms per check at the median.

Table 6: Chain audit in the extension campaign. Targets: interval assets that only chains of existing intervals tile; R1 rows: decompositions that R1 emits for them. Audited: interval assets in one audit pass, with its median time in ms. Detected: attacks (budgets 10%, 25%, 50%) whose violated chain the audit flags. Parts: parts of the cheapest flagged chain. Rounds: lazy-enforcement rounds until the audit flags nothing. Time: median / largest total time per attack in ms.

Dataset	Targets	R1 rows	Audited	Audit	Detected	Parts	Rounds	Time
NYC Taxi	60	0	1,369	22.6	180/180	3–4	2–5	220 / 468
TPC-H SF1	60	0	1,331	18.2	180/180	3–5	2–4	219 / 358
UCI Adult	60	0	1,389	22.0	180/180	3–4	2–4	225 / 316

5.4 Price floors

Table 7 reports the floor study. Floors of 10^{-6}, 10^{-4}, and 10^{-2} units raise 1,524, 6,272, and 7,033 list prices over the three datasets, and every floored list-price vector verifies at 10^{-12}. The three families are retrained on each floored policy, and the gate runs their 45 proposals at three tolerances: all 405 gate calls release without fallback, within 63 sweeps. Every leftover saving lies within the bound $\varepsilon/\ell_{\min} = \varepsilon/f$ of Corollary 6.4. At 10^{-7}, a floor of 10^{-6} units still leaves 30 of 45 releases above 1% and all 45 above 0.1%, a floor of 10^{-4} units keeps every saving at or below 0.09996%, under its bound of 0.1%, and a floor of 10^{-2} units keeps every saving below 0.001%. With one unit set to USD 2,176, the median price of the one-off data products among 4,162 priced products from 443 providers on commercial data marketplaces [2], the floors are USD 0.0022, USD 0.22, and USD 21.76.

Table 7: Price floors in the extension campaign: 45 retrained proposals per cell, gated against list prices $\max(\ell_i, f)$. Dollars set one price unit to USD 2,176. Threshold columns count releases that keep a saving above 1% or 0.1% of the target's floored list price. Max: largest saving. Bound: $\varepsilon/\ell_{\min}$ of Corollary 6.4. No cell used fallback.

Floor f	Dollars	ε	Released	Sweeps	>1%	>0.1%	Max	Bound
10^{-6}	USD 0.0022	10^{-7}	45/45	29	30/45	45/45	9.994%	10%
10^{-6}	USD 0.0022	10^{-9}	45/45	43	0/45	0/45	0.09994%	0.1%
10^{-6}	USD 0.0022	10^{-12}	45/45	63	0/45	0/45	9.919×10^{-5}%	10^{-4}%
10^{-4}	USD 0.22	10^{-7}	45/45	29	0/45	0/45	0.09996%	0.1%
10^{-4}	USD 0.22	10^{-9}	45/45	43	0/45	0/45	9.995×10^{-4}%	0.001%
10^{-4}	USD 0.22	10^{-12}	45/45	63	0/45	0/45	9.994×10^{-7}%	10^{-6}%
10^{-2}	USD 21.76	10^{-7}	45/45	29	0/45	0/45	9.997×10^{-4}%	0.001%
10^{-2}	USD 21.76	10^{-9}	45/45	41	0/45	0/45	9.994×10^{-6}%	10^{-5}%
10^{-2}	USD 21.76	10^{-12}	45/45	61	0/45	0/45	9.976×10^{-9}%	10^{-8}%

5.5 Distance to the exact projection

Table 8 gives latency and price change on the 45 proposals, 15 per dataset; every output of every method passed verification at 10^{-12}. The bracket of Section 2.7 had a relative width $U/L - 1$ of at most 2.24×10^{-7}

and a median of 1.25×10^{-13}, and it took at most 3.06 s per proposal. Refined Clarabel duals gave the lower bound for 35 proposals, unrefined Clarabel duals for 5, and refined multipliers recovered from the OSQP solution for 5; blends of the OSQP solution toward a Clarabel solution gave the upper bound for 35 proposals, the OSQP solution itself for 5, and a blend between the two Clarabel solutions for 5.

Arbiter's L2 price change is a median 1.08, 1.03, and 1.11 times that of $\mathbf{p}^\star$ for the median predictor, gradient boosting, and the perceptron, 1.003 to 1.17 overall, and full cyclic repair's is 1.09, 1.02, and 1.08 times, 1.004 to 1.13 overall. Active-row repair therefore does not return $\mathbf{p}^\star$, as Section 5.2 of the paper states, but its price change is at most 1.17 times the minimum on every proposal. The median distance from Arbiter's output to $\mathbf{p}^\star$ is at most 0.032 units. OSQP's output lies within 6.2×10^{-5} units of $\mathbf{p}^\star$, with a median of 1.1×10^{-7}, although OSQP stops at its 20,000-iteration limit in 54 of 135 trials. Clarabel at its default tolerance stops with status Solved in all 135 trials, within 21 iterations, and its outputs lie a median 1.9×10^{-4} units from $\mathbf{p}^\star$. That deviation is small against most projection distances but not against those of the 10 median-predictor proposals of NYC Taxi and UCI Adult, 1.9×10^{-5} and 5.6×10^{-6} units, where Clarabel's change is 2.39 and 6.70 times the minimum. At a gap tolerance of 10^{-12}, Clarabel stops with insufficient progress in 105 of 135 trials, within 29 iterations, and returns vectors that pass verification but change prices a median 1.84 and at most 2,238 times as much as $\mathbf{p}^\star$. The L1 norm of Arbiter's price change is a median 1.06 times that of OSQP, with a range of 1.02 to 1.11, as in the revision campaign.

The paired speedups of Arbiter on NYC Taxi, TPC-H SF1, and UCI Adult are 4.21 [3.04, 4.26], 2.12 [1.42, 5.70], and 19.0 [18.8, 32.4] over Clarabel at its default tolerance; 2.75 [2.01, 3.07], 1.76 [1.72, 4.49], and 23.7 [14.3, 25.0] over Clarabel at 10^{-12}; 6.50 [3.44, 6.74], 105 [3.73, 292], and 51.8 [2.13, 237] over OSQP; and 11.6 [11.3, 18.6], 17.6 [9.88, 33.0], and 15.3 [8.77, 30.9] over full cyclic repair. OSQP took up to 8.04 s on these proposals.

Table 8: Model proposals in the extension campaign at tolerance 10^{-12}: median / p95 latency in ms over 3 timed repetitions per proposal, and L2: median ratio of the L2 price change to that of the exact projection $\mathbf{p}^\star$. Clarabel rows give the solver tolerance. Every output passed verification.

Method	NYC Taxi	TPC-H SF1	UCI Adult	L2
Arbiter	43.6 / 82.9	57.2 / 73.8	50.1 / 69.8	1.07
Full cyclic	808 / 1,018	703 / 1,165	870 / 1,073	1.07
OSQP, 10^{-12}	287 / 7,258	5,931 / 6,025	2,897 / 7,986	1.00
Clarabel, 10^{-8}	182 / 189	114 / 121	1,028 / 1,094	1.00
Clarabel, 10^{-12}	133 / 140	90.9 / 126	789 / 1,271	1.84

5.6 Scale-out

The audited generator produces 8,192 assets per dataset and 12,288 assets on TPC-H SF1 and UCI Adult. At 4,096 assets it reproduces the audited workloads, relations, and snapshot digests exactly, and the compiler matches the oracle with precision and recall 1.0 on every workload. Two combined workloads join the three datasets' snapshots at 4,096 and 8,192 assets each. Every method receives the same exact per-row attack candidates with a budget of 25%: four per relation family and dataset, 12 per workload, and two per family and dataset in the combined workloads, 18 per workload. The candidates violate 3 to 81 rows. Table 9 gives every workload. Arbiter's median is 0.97 ms on the combined 12,288-asset workload and 1.93 ms on the combined 24,576-asset workload, whose snapshot holds 62,826 relations in 87,402 rows with 163,926 nonzeros. The largest p95, 9.77 ms, is that of the largest workload, and Arbiter needs at most 30 sweeps on any candidate. Every output of every method passed verification.

6 Full Comparison of Pricing Approaches

Table 1 of the paper is a shortened form of Table 10, which keeps four further approaches and two further columns. The citations for each approach are those of the paper.

Table 9: Scale-out in the extension campaign at tolerance 10^{-12} on exact per-row attack candidates (12 per dataset workload, 18 per combined workload; budget 25%). Build: snapshot build time in ms (median of three builds; one build for combined workloads). p50 and p95: Arbiter's latency in ms over 10 timed repetitions after 3 warm-ups. Speedups: paired, with 95% bootstrap intervals, over full cyclic repair, OSQP at $\epsilon_{\text{abs}} = 10^{-12}$, and Clarabel at its default tolerance, each timed over 3 repetitions after one warm-up. Every output of every method passed verification. Combined workloads join the three datasets' snapshots at 4,096 and 8,192 assets each.

Workload	Assets	Relations	Rows	Build	p50	p95	Full cyclic	OSQP	Clarabel
NYC Taxi 4,096	4,096	9,447	13,543	154	0.42	0.56	184 [87.6, 665]	168 [141, 175]	389 [358, 406]
NYC Taxi 8,192	8,192	20,230	28,422	343	0.74	0.87	109 [106, 176]	218 [186, 331]	483 [456, 508]
TPC-H SF1 4,096	4,096	9,200	13,296	155	0.41	0.50	88.1 [83.6, 1,290]	168 [125, 310]	233 [220, 247]
TPC-H SF1 8,192	8,192	21,494	29,686	364	0.77	4.14	111 [109, 272]	311 [128, 349]	397 [235, 405]
TPC-H SF1 12,288	12,288	34,289	46,577	585	1.18	5.85	122 [117, 348]	377 [146, 597]	390 [234, 422]
UCI Adult 4,096	4,096	9,850	13,946	173	0.46	0.60	83.8 [77.8, 342]	195 [171, 588]	2,011 [1,692, 2,064]
UCI Adult 8,192	8,192	21,102	29,294	356	0.77	0.90	110 [106, 454]	296 [242, 382]	591 [563, 613]
UCI Adult 12,288	12,288	34,879	47,167	614	1.16	1.39	118 [115, 941]	541 [316, 2,128]	2,969 [2,924, 3,037]
Combined 3×4,096	12,288	28,497	40,785	504	0.97	1.15	122 [117, 240]	347 [246, 620]	546 [534, 549]
Combined 3×8,192	24,576	62,826	87,402	1,128	1.93	9.77	130 [128, 260]	474 [423, 903]	635 [628, 649]

References

1. S. Agmon. The Relaxation Method for Linear Inequalities. *Canadian Journal of Mathematics*, 6:382–392, 1954. doi:10.4153/cjm-1954-037-2.

2. S. A. Azcoitia, C. Iordanou, and N. Laoutaris. Understanding the Price of Data in Commercial Data Marketplaces. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*, pages 3718–3728, 2023. doi:10.1109/ICDE55515.2023.00300.

3. H. H. Bauschke and J. M. Borwein. On Projection Algorithms for Solving Convex Feasibility Problems. *SIAM Review*, 38(3):367–426, 1996. doi:10.1137/S0036144593251710.

4. P. Goulart. Clarabel, Version 0.11.1. Rust and Python package, Apache License 2.0, June 2025. `https://github.com/oxfordcontrol/Clarabel.rs`, accessed October 24, 2026.

5. C. Hildreth. A Quadratic Programming Procedure. *Naval Research Logistics Quarterly*, 4(1):79–85, 1957. doi:10.1002/nav.3800040113.

6. A. J. Hoffman. On Approximate Solutions of Systems of Linear Inequalities. *Journal of Research of the National Bureau of Standards*, 49(4):263–265, 1952.

Table 10: Scope and guarantees of pricing approaches. Pricing theory checks the seller's own price points for consistency when the seller defines them. Only Arbiter checks every supplied vector at release and issues a release token.

Approach	Priced object or scope	Native construction	History state	Primary property	Token
Pricing theory	Queries over priced views	Unique extension of view prices	Not central	Arbitrage and discount freeness	No
QueryMarket	Conjunctive join queries	Integer program over views	Purchase history	Information-aware price	No
QIRANA	Broad SQL with aggregates	Support sets, weighted coverage	Yes	Arbitrage-free price function	No
Revenue pricing	Seller query workload	Constrained revenue optimization	Market sequence	Seller revenue	No
PRIDE	Repeated feature-based offers	Online value and discount learning	Temporal state	Revenue and regret	No
ARIA	Select-project-join and simple aggregates	Tuple information gain	Not reported	Scalable arbitrage freedom	No
Dealer	Models at chosen privacy levels	Model menu and compensation	Market state	Revenue, privacy, Shapley	No
DataPrice	Complete dataset listings	Metadata-based recommendation	No	Price recommendation	No
Arbiter	Single-table SQL fragment with one-column `GROUP BY`	None; checks a supplied price	Per-buyer snapshot	Encoded relations hold	Yes